

前置学习内容——动态内存分配

前言

在前面我已经学会了C语言中的指针操作，至少已经知道了一个指针变量中存储的是什么样的东西。指针提升了我们内存操作的灵活性，让我能够随心所欲去修改/解释内存。但我们都知道，答题卡总是规定了我们书写的边界，哪怕我们书写得再灵活（改卷子的老师能不能看懂我就知道了）也不能离开答题区域进行书写。

在我们的C语言程序中，这个规定了我们答题区有多大的东西就是 `栈`。这里我们不讲栈这样的数据结构，而是仅仅将其视作一片可供程序使用的内存区域。当我们定义的变量太多，多到答题卡不足以容纳下我们的结果，就会导致栈溢出（`stack overflow`）。但有时候我们“确信已找到了一个极佳的证明，但书的空白太窄，写不下”，这要怎么办呢？我们有三种可选的办法：

1. 优化算法
2. 扩大答题卡
3. 使用答题区以外的空间，甚至把草稿纸也交上去

这里我们要使用的就是第三种方案——`malloc`。

malloc

与普通变量使用 `栈` 内存区域作为存储空间不同，我们可以使用 `malloc` 来获取另一个区域的内存——堆内存。堆内存并不像栈一样有限（相对），那是一片庞大无数倍、更加自由的内存区域，使用堆内存时程序员几乎不需要担心它不够使用。

我们首先学习如何使用 `malloc`：

`malloc` 在 `stdlib.h` 中被声明，使用前需引用该头文件。

代码块

```
1 #include <stdlib.h>
```

以下则是它的声明：

代码块

```
1 void *malloc(size_t size);
```

其返回值类型为 `void*`，可见 `malloc` 是个指针函数，其返回的是指向 被分配内存的起始位置 的指针，不过是 `void*` 类型的指针。其往后的 `size` 大小字节的内存是已分配且未初始化的。

如果你对 `void*` 较为感兴趣，可以通过[此链接跳转](#)学习。

`void*` 类型表示其指向的那段内存将可以当做任何类型的变量进行处理，不过为了更安全地使用 `malloc` 返回的内存指针，我们最好还是进行一个强制转换，比如将 `void *` 转换为 `int *`，我们给出如下代码：

代码块

```
1  int *alloc_arr = NULL;
2  alloc_arr = (int*)malloc(10*sizeof(int));
3  *(alloc_arr + 0) = 9;
4  printf("%d\n", *alloc_arr);
```

这里为 `alloc_arr` 指针赋值了一个堆内存的地址，让其指向一段已分配的堆内存。这时候就可以把 `alloc_arr` 当做指向数组首元素的指针来使用了，只是这个数组只有10个成员，而且 `alloc_arr` 中也不包含成员个数的信息（这在我们第二周讲义中曾作为支线任务进行提示）。

不过由于堆中的数据没被初始化，这时候访问到数组内的元素谁也不知道里面会是什么（虽然有不小概率会是0）。因此我们在使用这段内存前必须进行初始化以避免程序产生不符合预期的行为。

这之后对这段内存的使用，就和前面使用指针并没有什么太大的不同了。

不过话又说回来，虽然我们现在能离开答题区域进行作答，但批卷子的老师却并不希望我们的有效答案分布于答题区外的某个不知名角落里。而在C语言程序中，这个老师就是我们的操作系统。

free

当我们不小心把有效答案写出了答题区域，我们的第一反应一定是用一堆似是而非的记号来告诉改卷老师“我的答案在这里”（PS：别这么干，还是换一张答题卡吧）。

众所周知，类比总是不完美的，操作系统这个改卷老师就和现实里的不同，它为所有写C语言程序的程序员提供了一个专用标记——`free` 函数。同样不同的是，程序员使用这个标记也不是告诉操作系统这里是答案，而是说“我的答案不在这里”。

依旧是刚刚的例子，我们需要增加一个 `free` 函数，以下是 `free` 函数的声明和使用：

代码块

```
1  void free(void *ptr);
```

代码块

```
1  int *alloc_arr = NULL;
```

```
2  alloc_arr = (int*)malloc(10*sizeof(int));
3  *(alloc_arr + 0) = 9;
4  printf("%d\n", *alloc_arr);    // 此时已经完成了这段堆内存的任务，是时候结束它了
5
6  free(alloc_arr);
7  alloc_arr = NULL;
```

`free` 函数的使用是如此简单，以至于我们不需要太多的解释。但有一点必须要注意的是，`free` 传入的 `ptr` 参数不能是空指针，也不能是指向非 `malloc` 返回的地址。

因此，我们在 `free` 前需要进行一次判断：

代码块

```
1  int *alloc_arr = NULL;
2  alloc_arr = (int*)malloc(10*sizeof(int));
3  *(alloc_arr + 0) = 9;
4  printf("%d\n", *alloc_arr);
5  if(alloc_arr != NULL){
6      free(alloc_arr);
7  }
8  alloc_arr = NULL;
```

由于我们难以判断传入的指针是否来自 `malloc`，所以一般也不进行此类判断。

危险的内存泄漏

你不经要问：如果我总是不进行`free`会如何呢？要是继续沿用前面的类比，你的考试成绩最终就是0，而在我们的C语言程序中，这样的后果就是内存泄漏。

这里我们仅介绍两种常见的内存泄漏：

未free

虽然前面我们提到——堆内存非常庞大，但这并不意味着我们有限大小的计算机真的有无限大小的堆内存可供我们使用。一般情况下，造成我们堆内存短缺的主要原因并不是堆内存太小，而是产生了内存泄漏。

与纸面材料写上了就不能再重复利用不同，我们使用 `free` 后释放的这段堆内存将会是可重复利用的（即回到可被 `malloc` 分配的状态）。一般情况下程序是使用不完如此庞大的堆内存的，这就是得益于 `free` 函数。但假如我们一直不进行 `free`，那么这段内存将一直在使用中，直到我们的程序运行失败，这通常见于大型项目和服务器。

悬空指针

和以上会造成堆内存短缺的情况不同，这样的内存泄漏后果通常更加严重也更加令人费解。

在上文的[示例代码](#)中，我们在 `free` 后使用 `alloc_arr = NULL`；将 `alloc_arr` 设置为 `NULL`，就是为了避免产生悬空指针。这也是上一次讲义中 `FILE*` 指针 `fp` 为何要在 `fclose` 后需要设置为 `NULL` 的原因。

我们传入 `free` 的参数是 `void *` 类型的指针 `ptr`，这样就注定它无法修改我们的 `ptr` 指针本身，于是悬空指针就此产生。如果在 `free` 后继续使用该段内存，将会产生无法预料的后果，包括但不限于：

- Segment Fault
- 数据损坏或被错误地更改（比如这段内存被重新分配，而它的旧主人仍在使用这段内存）
- 难以debug
-

约定

内存泄漏是如此严重，所以针对动态内存分配（`malloc` 类函数）我们需要给出如下五个约定：

1. 对于由 `malloc` 返回的任何指针，存储它的指针变量必须有至少一个在 `free` 前始终不变（但不能是指针常量）。
2. 每次获取 `malloc` 分配的内存后，必须使用 `if` 语句判断 `malloc` 的返回是否为 `NULL`。
3. 每次 `free` 前，必须使用 `if` 语句判断传入的参数值是否为 `NULL`。
4. 对同一段由 `malloc` 分配的堆内存的释放，`free` 函数传入的参数只能是以上定义的其中一个指针，而不能是其他。
5. `free` 后必须使用 `NULL` 清除**所有**悬空指针。

牢记约定

或许你并没有学习前面长篇的知识，但这些约定你无论如何都必须遵守。后面的大作业中我们会检查你是否遵守了这五个约定。

在大作业后你也同样需要遵守这五个约定，直到你能够熟练使用 `malloc`，至少在你知道哪些约定相对冗余前。

何为动态

我们给出如下示例代码：

代码块

```
1  #include <stdlib.h>
```

```

2  #include <stdio.h>
3  int main(void){
4      int *ptr = NULL;
5      ptr = (int *)malloc(10*sizeof(int));
6      if(ptr == NULL) { printf("error in malloc\n"); return -1; };
7
8      for(int i = 0; i < 10; i++){
9          *(ptr + i) = i;
10     }
11     for(int i = 0; i < 10; i++){
12         printf("%d ", *(ptr + i));
13     }
14     printf("\n");
15
16
17     if(ptr != NULL){
18         free(ptr);
19     }
20     ptr = NULL;
21
22     return 0;
23 }

```

这样，我们就实现了在堆中开辟一段数组并赋值输出的需求。

为了方便接下来的讲义阅读，你需要保证你已经 **RTFSC** 并理解代码的行为。

在你阅读完这个代码并运行后，你有这样一个发现——这似乎与我们平时声明的数组并没有什么区别，仅仅是这个数组储存存在于堆内存区域。这是因为我们在上文仅仅讲述了如何使用 **malloc** 进行 **分配**，并没有向你讲如何实现动态内存分配中的 **动态**。

接下来我们就需要提到堆内存的动态性质，我们先确定一个情景，这将建立在你 **RTFSC** 之后。

在你输出了全部从 **0** 到 **9** 的整数后，你想要让程序再从 **0** 输出到 **1000**，并在这之前用 **一个数组** 储存它们，这该怎么做？我们有三种方案：

1. 程序好蠢，我要自己手写出这 **990** 个数字
2. 更改代码，把分配的大小改成 **1000**，然后新写一个循环并把 **10** 改成 **1001**
3. 使用动态内存分配的动态方法

尽管第二种方法在这样简单的情况下是适用的，但要是情况变得复杂就不那么方便了。因此我们使用第三种方案，我们给出如下丑陋的代码（默认已折叠）：

代码块

```

1  // 此处为未使用函数且无注释的的初版代码
2  #include <stdlib.h>

```

```
3  #include <stdio.h>
4  int main(void){
5      int *ptr = NULL;
6      ptr = (int *)malloc(10*sizeof(int));
7      if(ptr == NULL) { printf("error in malloc of ptr\n"); return -1; };
8
9      for(int i = 0; i < 10; i++){
10         *(ptr + i) = i;
11     }
12     for(int i = 0; i < 10; i++){
13         printf("%d ", *(ptr + i));
14     }
15     printf("\n");
16
17     int num = 0;
18     printf("input a num bigger than 9: ");
19     scanf("%d", &num);
20
21     int *ptr2 = (int *)malloc((num + 1)*sizeof(int));
22     if(ptr2 == NULL) { printf("error in malloc of ptr2\n"); return -1; };
23
24     for(int i = 0; i < 10; i++){
25         *(ptr2 + i) = *(ptr + i);
26     }
27     if(ptr != NULL){
28         free(ptr);
29     }
30     ptr = NULL;
31
32
33     for(int i = 10; i < num + 1; i++){
34         *(ptr2 + i) = i;
35     }
36     for(int i = 0; i < num + 1; i++){
37         printf("%d ", *(ptr2 + i));
38     }
39     printf("\n");
40
41     if(ptr2 != NULL){
42         free(ptr2);
43     }
44     ptr2 = NULL;
45
46     return 0;
47 }
```

为了方便地让大家 `RTFSC`，我们选择使用函数的方法来简化整个流程，并添加了部分注释：

代码块

```
1  #include <stdlib.h>
2  #include <stdio.h>
3
4  // 输出right个数组元素
5  void print(int *ptr, int right){
6      for(int i = 0; i < right; i++){
7          printf("%d ", *(ptr + i));
8      }
9      printf("\n");
10 }
11
12 // 实现更好的free
13 void new_free(void **ptr_){
14     if(ptr_ != NULL && *ptr_ != NULL){
15         free(*ptr_);
16         *ptr_ = NULL;
17     }
18 }
19
20 // 设置数组的值
21 void set_array(int *ptr, int left, int right){
22     for(int i = left; i < right; i++){
23         *(ptr + i) = i;
24     }
25 }
26
27 int main(void){
28
29     // 设置储存0到9这10个数字的数组
30     int *ptr = NULL;
31     ptr = (int *)malloc(10*sizeof(int));
32     if(ptr == NULL) { printf("error in malloc of ptr\n"); return -1; };
33     set_array(ptr, 0, 10);
34     print(ptr, 10);
35
36     // 完成输出后，我们想要用户输入一个数字，并输出0到num的所有整数
37     int num = 0;
38     printf("Please input a num bigger than 9: ");
39     scanf("%d", &num);
40     if(num <= 9) {
41         printf("error in usr's input: need a num bigger than 9\n");
42         return -1;
43     }
```

```

44
45 // 设置储存0到num这num+1个数的数组，同时转移数据和释放空间
46 int *ptr2 = NULL;
47 ptr2 = (int *)malloc((num + 1) * sizeof(int));
48 if(ptr2 == NULL) { printf("error in malloc of ptr2\n"); return -1; };
49 for(int i = 0; i < 10; i++){
50     *(ptr2 + i) = *(ptr + i);
51 }
52 new_free((void **)&ptr);
53
54 // 转移完成后，将剩余需要写入的数据写入
55 set_array(ptr2, 10, num + 1);
56 print(ptr2, num + 1);
57 new_free((void **)&ptr2);
58 return 0;
59 }
60

```

这样的代码不仅实现了从 0 到 1000 的输出和存储（你可以将其存入文件），而且还可以是任意的数字。

通过 RTFSC 可以发现，我们在需要一个新数组时，选择用一个新的 `malloc` 来进行额外分配，而前面旧的 `ptr` 则在其数据转移成功后进行 `free`。这样一来，我们就完成了前面提到的功能，而且没有使用数组，仅仅依靠指针来完成。

而为了更好地像以上代码一样进行重新分配，C语言还有一个名为 `realloc` 的库函数。如果你愿意的话，也可以使用 `realloc` 来进行功能替换，但我更希望在此之前你能学习 `realloc` 重新分配内存的行为和以上代码的行为有什么不同。



（支线）RTFSC

为了检验你是否真的完成了RTFSC，你需要更改以上代码，并用一个新的函数来代替我们使用的数据转移。要求如下：

1. 函数声明为 `void intArrayCopy(int *ptr_dest, int *ptr_source, int left, int right);`
2. 模仿以上代码中声明为 `void set_array(int *ptr, int left, int right);` 的函数，完成数据的转移。
3. 写上函数注释，并替换代码中的数据转移操作（是一个 `for` 循环）
4. 为了方便批改。你需要将你的函数体放在 `main` 函数之后，而在 `main` 开始前放置函数声明。

完成后，将你更改后的代码保存并重命名为 `exercise` 并与你的大作业一同提交。

void*

`void *` 指针类型在我们的man手册中被这样描述：

man 3 void (有删改)

```
1 void *
2     According to the C language standard, a pointer to any object type may be
    converted to a pointer to void and back. POSIX further requires that any
    pointer, including pointers to functions, may be converted to a pointer to
    void and back.
3
4     This type is useful in function parameters and return value to allow
    passing values of any type. The function will typically use some mechanism to
    know the real type of the data being passed via a pointer to void.
5
6     A value of this type can't be dereferenced, as it would give a value of
    type void, which is not possible. Likewise, pointer arithmetic is not possible
    with this type.
7
8     However, in GNU C, pointer arithmetic is allowed as an extension to the
    standard; this is done by treating the size of a void or of a function as 1. A
    consequence of this is that sizeof is also allowed on void and on function
    types, and returns 1.
```

我们对其进行删减，只展示其中你可能用上或理解的概念。以下是翻译：

代码块

```
1 void *
2     根据 C 语言标准，任何对象类型的指针都可以转换为 void 类型的指针 (void*)，然后再转换
    回来。POSIX则进一步要求，任何指针（包括函数指针）都可以转换为 void 类型的指针，然后再转换
    回来。
3
4     这种类型在函数参数和返回值中很有用，可以传递任何类型的值。函数通常会使用某种机制来确定
    通过 void 类型的指针传递的实际数据类型。
5
6     这种类型的值不能进行解引用，因为它会给出 void 类型的值，这是不可能的。同样，这种类型
    不支持指针算术。
7
8     然而，在 GNU C 中，指针运算是作为标准的一个扩展而允许的；这是通过将 void 或函数的大
    小视为 1 来实现的。这一后果是 sizeof 也允许用于 void 和函数类型，并返回 1。
```

[看完了就回去吧^_^](#)